

Лабораторна робота №2

Контейнеризація

Мета роботи

Метою завдання є практичне закріплення знань з таких тем курсу:

- Основи контейнеризації
- Використання docker для упаковки застосунку
- Використання docker compose для запуску програмної системи, що складається з декількох сервісів

Студент повинен провести дослідження щодо використання контейнеризації для упаковки застосунку та підготувати автоматичний запуск усіх сервісів із [Лабораторної роботи №1](#). в контейнерах.

Завдання

Опис завдання

Лабораторна робота складається з двох основних частин: дослідницької та практичної. Дослідницька частина полягає в тому, щоб дослідити різні підходи контейнеризації, виявити слабкі та сильні сторони кожного з підходів.Arteфактом дослідницької частини має бути звіт, який детально описує виконані дослідження, спосіб їх відтворення, їх результати та зроблені висновки.

Практична частина полягає у практичному застосуванні отриманих знань та навичок для ефективної контейнеризації сервісів, необхідних для роботи застосунку із Лабораторної роботи №1. Arteфактом практичної частини має бути файл docker-compose.yml в репозиторії з виконанням першої лабораторної роботи. Також студенти мають доповнити звіт дослідницької частини розділом, який описує які кроки було виконано для контейнеризації набору сервісів.

Дослідницька частина

Python Application

Працюємо із проектом на Python із репозиторія:

<https://github.com/KPI-FICT-MTSD/lab-03-starter-project-python>

1. Спершу створюємо опис контейнера для застосунку. Додаємо в цей образ код застосунку, залежності¹, та збираємо наш образ. Необхідно виміряти:
 - розмір образу
 - час збірки образу²
2. Після цього, вносимо зміни у файл [build/index.html](#) або [spaceship/app.py](#) (додаємо коментар, заставляємо програму вивести ваші ім'я та прізвище, абощо) і повторно збираємо образ із новою версією коду. Вимірюємо:
 - розмір образу
 - час збірки образу
3. Переписуємо файл опису образу (dockerfile) таким чином, щоб ефективно використовувати шари (layers) для збірки образу. Для цього спершу додаємо у образ те, що змінюватиметься найрідше (системні залежності, залежності проекту) а ті файли, які змінюватимуться частіше (наприклад, наш код) — додаємо останніми. Після цього повторюємо експеримент із п.2. (якщо ви з самого початку написали ідеальний dockerfile, то напишіть цього разу неідеальний, і порівняйте результати).
4. Візьміть за основу проекту менший базовий образ. Наприклад, замість [python:3.13-trixie/bookworm](#)³ (на базі debian) спробуйте [python:3.13-alpine](#) (на базі alpine). Виміряйте розмір фінального образу та час його збірки.
5. Додайте у проект залежність [numpy](#), та додайте в [spaceship/routers/api.py](#) простий ендпоінт, який за допомогою доданої залежності згенерує 2 випадкові матриці 10x10 та [перемножить](#) їх між собою (нам же треба якось виправдати додавання нової залежності в проект). Ендпоінт повинен повертати словник наступної структури:

```
{
    "matrix_a": [[...], ...],
    "matrix_b": [[...], ...],
    "product": [[...], ...],
}
```

Порівняйте час збірки та розміри нового образу із numpy для образів на основі alpine та debian.

Musl (Alpine) vs glibc (Debian/Ubuntu/etc)

1. Створюємо окрему мережу в docker для тестів:

```
$ docker network create dns-lab
```

¹ В проекті на GitHub вказано лише основні залежності проекту. Ваша ж задача зробити збірку образу максимально відтворюваною, тому доведеться подумати, як це зробити так, щоб при виході нових версій бібліотек, що використовуються у проекті, ці версії не зтягувалися автоматично у docker образ без нашого відомо. Найпростішим способом є закріплення (пінінг) версій усіх залежностей. В Python-світі багато хто використовує команду `pip freeze` для фіксування списку залежностей і використовує згенерований нею список для того, щоб встановлювати завжди одні і ті ж версії пакетів бібліотек. Але є і інші способи.

² При замірюванні часу збірки образу враховуйте те, що базовий образ скоріш за все ви завантажуватимете тільки один раз, в той час як решта етапів будуть виконуватися постійно. Для точності вимірювань або завантажте базовий образ перед збіркою окремою командою `git pull`, щоб час на його завантаження не враховувався, або видаляйте його після кожної збірки щоб в кожній збірці він завантажувався наново.

³ По можливості використовуйте останню стабільну версію python замість 3.13

2. Запускаємо простий dns-сервер, який вміє резолвити наш кастомний домен

```
$ docker run --rm -it --name dns-server --network dns-lab \
  alpine sh -c "apk add dnsmasq && \
  echo 'address=/myservice.internal.corp/10.0.0.50' > /etc/dnsmasq.conf && \
  dnsmasq -k --log-queries --log-facility=-"
```

3. В новому вікні запускаємо контейнер з ubuntu, в якому пробуємо розрезолвити наш домен

```
$ docker run --rm --network dns-lab \
  --dns=$(docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' dns-server) \
  --dns-search="corp" \
  ubuntu:latest getent hosts myservice.internal
```

4. В новому вікні запускаємо контейнер з alpine

```
$ docker run --rm --network dns-lab \
  --dns=$(docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' dns-server) \
  --dns-search="corp" \
  alpine:latest getent hosts myservice.internal
```

5. Порівняти результати кроків 3 і 4, проаналізувати логи DNS сервера із п.1. та пояснити отримані результати. У висновках зробити припущення, до чого така поведінка системи може призводити.

Golang Application та Multi-stage builds

Працюємо із проектом, що знаходиться в репозиторії

<https://github.com/comsys-kpi-ua/deploy.lab-containers-starter-project-golang>.

1. Створюємо образ проекту. Додаємо в контейнер залежності, збираємо виконуваний файл, тощо. Заміряємо час збірки проекту та розмір результуючого образу. Аналізуємо вміст нашого образу. Чи усі файли, що там є, потрібні для запуску проекту?
2. Робимо багатоетапну збірку. Для цього на першому етапі збираємо двійковий файл, а на наступному — копіюємо цей зібраний файл у новий порожній образ ([FROM scratch](#)). Аналізуємо вміст образу⁴. Чи достатньо файлів для запуску проекту⁵? Чи зручно таким образом користуватися? Чи зручно виконувати якісь дії всередині контейнеру, запущеного з цього образу?
3. Повторюємо п.2, але замість scratch використовуємо образи із проекту [distroless](#). Аналізуємо розміри образу та його вміст.

Практична частина

За допомогою [Docker Compose](#) автоматизувати запуск усіх 3 сервісів (web application, nginx, database) із [Лабораторної роботи №1](#). Сервіси повинні підніматися в окремій мережі (не в мережі за замовчуванням), дані БД повинні зберігатися на диску і переживати перезапуск контейнерів, видалення docker, перезавантаження системи.

⁴ Для аналізу вмісту образу можна скористатися утилітою [dive](#). Також за потреби можете використовувати будь-які інші інструменти.

⁵ Це нормально, якщо рішення "в лоб" не запуститься і буде видавати помилку "no such file or directory" або якусь іншу. Треба це описати, розібратися, чому виникає така помилка і описати рішення.

Результат роботи

Git репозиторій

Код ваших Dockerfile має бути розміщений на GitHub. Файл для Docker Compose має знаходитися в репозиторії лабораторної роботи №1. Репозиторій лабораторної роботи №1 доповнити документацією по запуску проєкту за допомогою Docker Compose.

Історія комітів повинна бути логічною та зрозумілою, відображати хід виконання роботи студентом.

Звіт

На основі проведених експериментів із Python-застосунком, Golang-застосунком та застосунком обраною вами мовою підготувати звіт. У звіті описати:

- що ви робили
- які результати (розміри/час збірки/тощо) отримали
- з якими особливостями/труднощами стикалися, які їх об'єктивні причини
- ваші висновки по отриманих результатах. На основі ваших висновків напишіть ваші рекомендації стосовно упаковки застосунків у контейнери.

Звіт ви готуєте у Google Docs / Word / або у вигляді веб-сторінки і завантажуєте документ або посилання на ваш звіт для здачі лабораторної роботи. У звіті ви не повинні дублювати прям весь код, натомість ви вказуєте тільки найважливіші фрагменти та робите посилання на конкретні файли у ваших репозиторіях, щоб можна було дізнатися більше.

Експерименти у звіті повинні бути відтворювані. Ви маєте вказати, як відтворити отримані вами результати (наприклад, цей експеримент запускався на такій-то конфігурації. Версія коду — така-то (лінк), осьо докерфайл (лінк), осьо команда для збору образу.

Важливо: не лити воду у звіті. Давайте збережемо один одному час і обійдемося без шаблонних "на цій лабораторній роботі я навчився(-лася) збирати docker-образи" і подібних фраз, які зазвичай просять вставити у звіт заради формальностей. Ви провели цілу низку експериментів. То розкажіть про них!